Rapport TP Final – Application Mobile Urban Explorer

Équipe :

Celia MERABET

Bassim Tabbeb

Mohamed Bouyabri

Abderrahmane Karim Rakem

Encadrant : Alexandre CADINOT

Année : 2025 – 2026

Table de matières :

1. Introduction 3
2. Architecture de l'Application 3
3. Intégration de l'API Open Data Paris 4
   3.1 API utilisée 4
   3.2 Gestion des requêtes 5
   3.3 Affichage des lieux 6
   3.4 Affichage de la liste avec FlatList 6
   3.5 Composant LieuCard 7
4. Carte et Géolocalisation 7
5. Planification de Visite 8
6. Caméra et Photo de Profil 8
7. Répartition des Tâches 8
8. Difficultés Rencontrées 9
9. Annexes 9

Année : 2025 – 2026

1. Introduction

Dans le cadre de ce projet de développement mobile, nous avons conçu une application appelée **Urban Explorer**.

L'objectif de cette application est de permettre aux utilisateurs de découvrir des lieux culturels et événements à Paris, de consulter leurs informations, de visualiser leur emplacement sur une carte et de planifier une visite.

L'application a été développée avec React Native et Expo, ce qui permet de créer une application mobile fonctionnant sur Android et iOS à partir d'un seul code.

L'application utilise également les données Open Data de la Ville de Paris, permettant d'afficher des informations réelles sur les événements et activités culturelles.

Les principales fonctionnalités de l'application sont :

- consultation des événements culturels à Paris
- affichage des lieux sur une carte interactive
- consultation des détails d'un événement
- planification d'une visite via un calendrier
- prise d'une photo de profil avec la caméra

2. Architecture de l'Application

L'application est organisée selon une architecture modulaire afin de faciliter la maintenance et la lisibilité du code.

Structure du projet :

assets/

src/

components/

- DetailsSkeleton.tsx
- LieuCard.tsx
- LieuCardSkeleton.tsx

contexts/

- VisitContext.tsx

navigation/

- AppNavigator.tsx

screens/

- CarteScreen.tsx
- DecouverteScreen.tsx
- DetailsScreen.tsx
- PlanningScreen.tsx
- ProfilScreen.tsx

services/

- api.ts

types/

- index.ts

Fichiers principaux :

- App.tsx
- package.json
- tsconfig.json

Cette architecture permet de séparer clairement :

- les écrans de l'application
- les composants réutilisables
- la gestion de l'API
- les types de données

Cela rend le projet plus facile à maintenir et à faire évoluer.

3. Intégration de l'API Open Data Paris

3.1 API utilisée

Pour récupérer les données des lieux culturels, nous avons utilisé **l'API Open Data Paris**, accessible via la console :

https://opendata.paris.fr/api/explore/v2.1/console

Le jeu de données utilisé est :

Que faire à Paris ?

L'endpoint utilisé dans l'application est :

https://opendata.paris.fr/api/explore/v2.1/catalog/datasets/que-faire-a-paris-/records

Cette API permet de récupérer des informations sur des **événements et activités culturelles à Paris**, telles que :

- le titre de l'événement
- l'adresse
- la description
- l'image associée
- les coordonnées géographiques

Dans l'application, les données sont récupérées sous forme de **JSON** et utilisées pour afficher une **liste dynamique de lieux culturels**.

3.2 Gestion des requêtes

L'appel à l'API est centralisé dans le fichier :

src/services/api.ts

Nous utilisons la bibliothèque **Axios** pour effectuer les requêtes HTTP.

La fonction principale utilisée est **fetchLieux**.

```
export const fetchLieux = async (): Promise<Lieu[]> => {
  const response = await axios.get(API_URL, {
    params: {
      limit: 30,
    },
  });

  return response.data.results.map((item: any) => ({
    nom_usuel: item.title || 'Lieu sans nom',
    adresse: item.address_name || item.address_street || 'Adresse non
disponible',
    coordonnees_geo: item.lat_lon
      ? {
          lat: item.lat_lon.lat,
          lon: item.lat_lon.lon,
        }
      : undefined,
  }));
};
```

Cette fonction :

- envoie une requête HTTP vers l'API Open Data Paris
- limite les résultats à **30 lieux**
- transforme les données reçues pour correspondre à la structure **Lieu** utilisée dans l'application

Cette transformation permet de garder une **structure de données cohérente** dans toute l'application.

3.3 Affichage des lieux

L'affichage des lieux est géré dans le fichier :

src/screens/DecouverteScreen.tsx

Lors du chargement de l'écran, l'application appelle la fonction **fetchLieux** grâce au hook **useEffect**.

```
useEffect(() => {
  const loadLieux = async () => {
    const data = await fetchLieux();
    setLieux(data);
  };

  loadLieux();
}, []);
```

Les données récupérées sont stockées dans un **état React** grâce à **useState** :

```
const [lieux, setLieux] = useState<Lieu[]>([]);
```

Pendant le chargement des données, l'application affiche des **skeleton loaders** grâce au composant :

LieuCardSkeleton

Cela améliore l'expérience utilisateur en simulant l'apparence des cartes avant que les données ne soient chargées.

3.4 Affichage de la liste avec FlatList

Une fois les données récupérées, les lieux sont affichés avec le composant **FlatList** de React Native.

FlatList est spécialement optimisé pour afficher des listes longues et améliorer les performances.

```
<FlatList
  data={lieux}
  keyExtractor={(item, index) => item.id || index.toString()}
  renderItem={({ item }) => (
    <LieuCard
      lieu={item}
      onPress={() => navigation.navigate('Details', { lieu: item })}
    />
```

```
  ) }
/>
```

Chaque élément de la liste est affiché à l'aide du composant **LieuCard**.

3.5 Composant LieuCard

Le composant **LieuCard** est un composant réutilisable situé dans :

src/components/LieuCard.tsx

Il permet d'afficher les informations principales d'un lieu culturel :

- l'image de l'événement
- le titre
- l'adresse
- une courte description

Pour l'affichage des images, nous utilisons la bibliothèque **expo-image**, qui permet un chargement plus performant des images.

Si aucune image n'est disponible, un **placeholder** est affiché.

Chaque carte est cliquable et permet d'accéder à l'écran **Details** grâce à la navigation **React Navigation**.

4. Carte et Géolocalisation

L'écran **CarteScreen** permet d'afficher une carte interactive grâce à la bibliothèque :

react-native-maps

Chaque lieu récupéré depuis l'API est affiché sous forme de **marqueur** sur la carte.

La carte est centrée sur **Paris** avec les coordonnées suivantes :

Latitude : 48.8566
Longitude : 2.3522

Lorsque l'utilisateur clique sur un marqueur, le nom du lieu s'affiche.

5. Planification de Visite

Dans l'écran **DetailsScreen**, l'utilisateur peut consulter les informations d'un lieu sélectionné.

Un bouton **Planifier une visite** permet d'ouvrir l'écran **PlanningScreen**.

Cet écran contient un calendrier utilisant la bibliothèque :

react-native-calendars

L'utilisateur peut sélectionner une date afin de planifier sa visite.

6. Caméra et Photo de Profil

L'écran **ProfilScreen** permet à l'utilisateur de prendre une photo de profil.

Pour cette fonctionnalité, nous utilisons la bibliothèque :

expo-image-picker

Lorsque l'utilisateur appuie sur le bouton **Prendre une photo**, l'application demande l'autorisation d'accéder à la caméra.

Une fois la photo prise, elle est affichée comme **photo de profil** dans l'application.

7. Répartition des Tâches

Le projet a été réalisé en équipe avec la répartition suivante :

Bassem

- structure du projet
- navigation de l'application

Mohamed

- intégration de l'API
- affichage des lieux

Celia

- carte interactive

- affichage des marqueurs

Karim

- calendrier de planification
- caméra et photo de profil

8. Difficultés Rencontrées

Durant le développement, plusieurs difficultés ont été rencontrées :

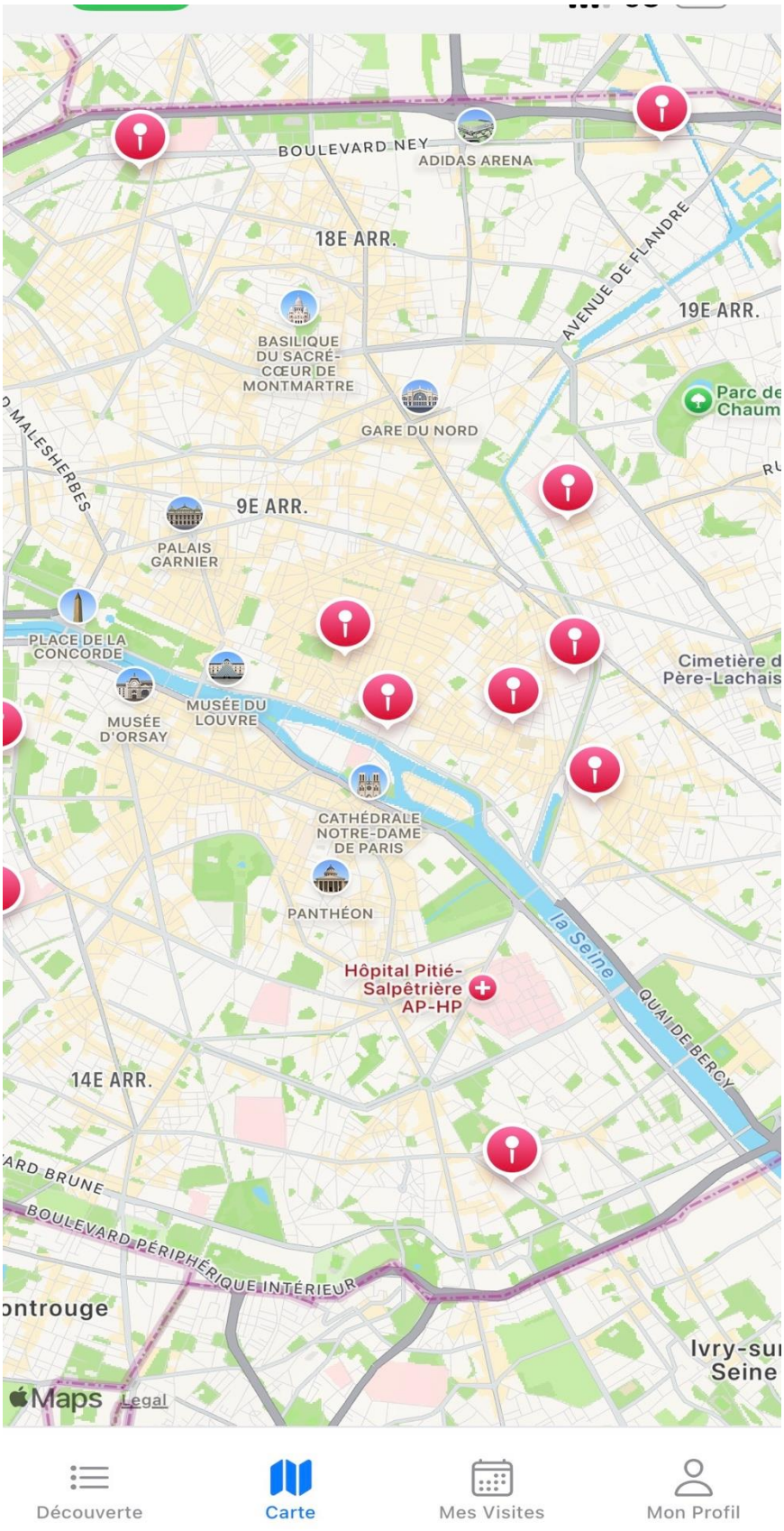
- gestion des données provenant de l'API
- correspondance entre les types TypeScript et les données JSON
- affichage correct des images
- gestion des permissions pour la caméra

Ces problèmes ont été résolus grâce à des vérifications supplémentaires dans le code et une meilleure gestion des données.

9. Annexes

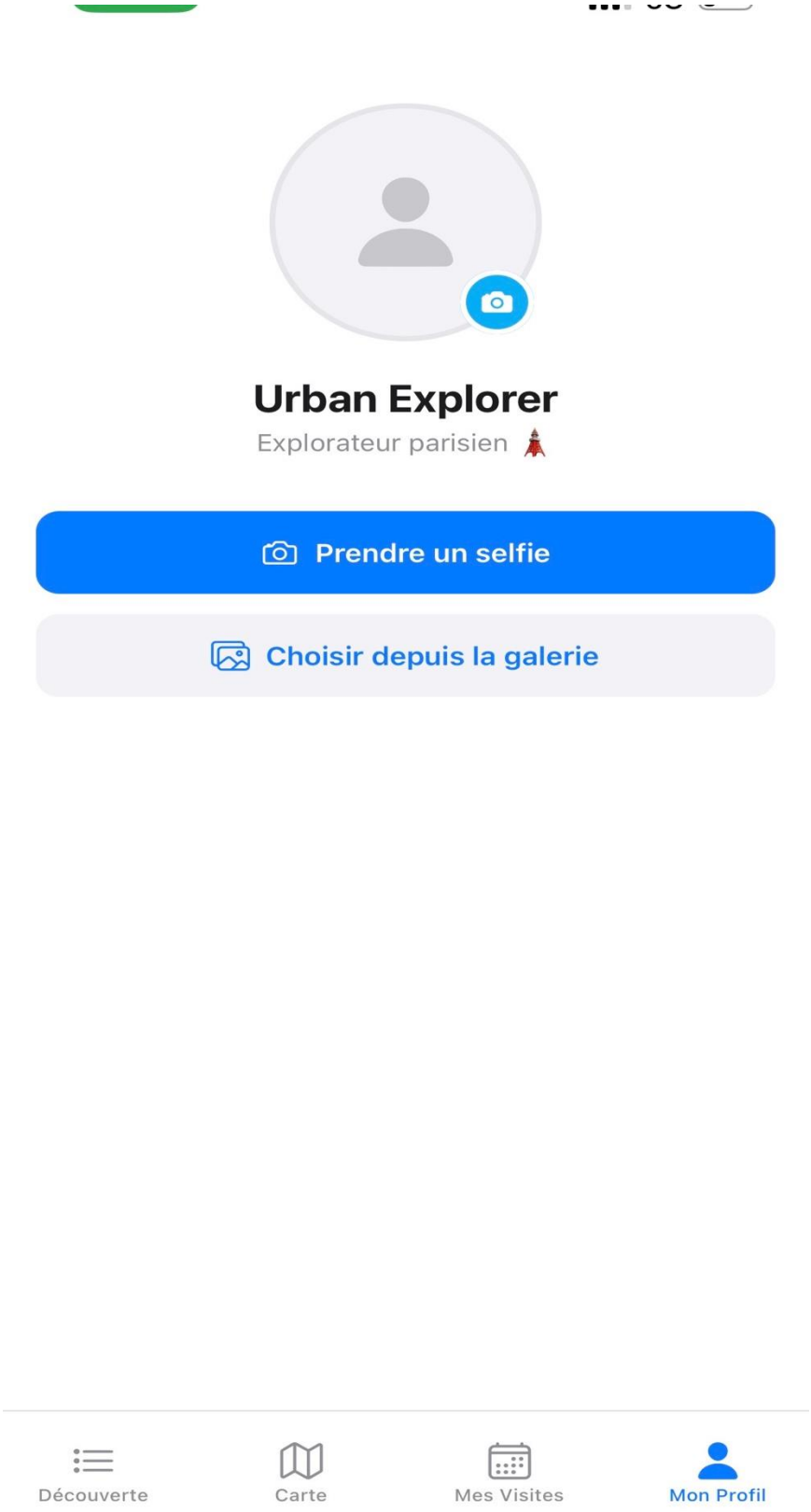
- capture d'écran de l'écran Découverte

- capture d'écran de la carte



- capture d'écran de la page de détails

- capture d'écran de la page profil



Année : 2025 – 2026

Mes Visites (1)

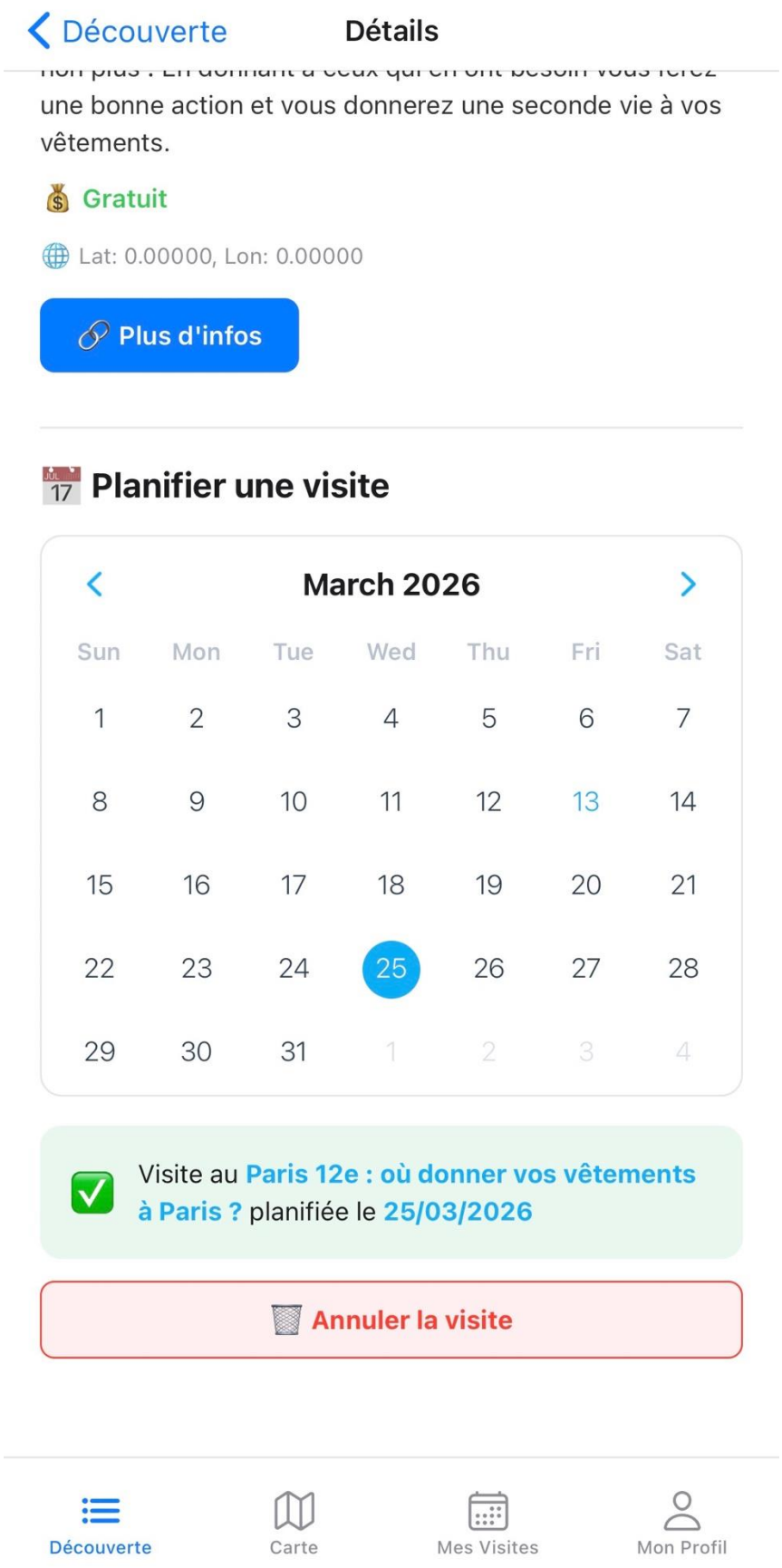
Paris 12e : où donner vos vêtements à Paris ?

📍 6-8 passage Raguinot

📅 25/03/2026

🗑️

Découverte | Carte | Mes Visites | Mon Profil


Découverte
Détails
une bonne action et vous donnerez une seconde vie à vos vêtements.
Gratuit
Lat: 0.00000, Lon: 0.00000
Plus d'infos
Planifier une visite
March 2026
Sun Mon Tue Wed Thu Fri Sat
1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31 1 2 3 4
Visite au Paris 12e : où donner vos vêtements à Paris ? planifiée le 25/03/2026
Annuler la visite
Découverte
Carte
Mes Visites
Mon Profil